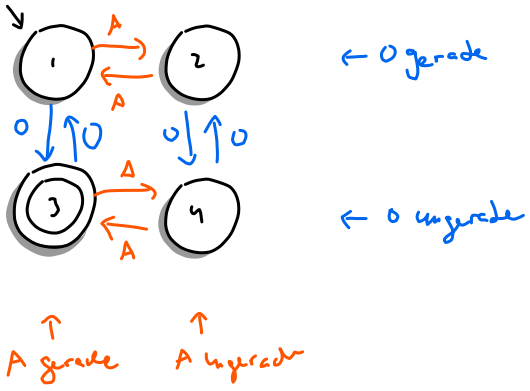
TheoInf 1
Woche 6

Lösungen zu der Hausübung 1

Im Folgenden verwenden wir das Alphabet Σ = {A, O}.
Σ* sei die Menge aller Strings, die aus Zeichen aus Σ bestehen

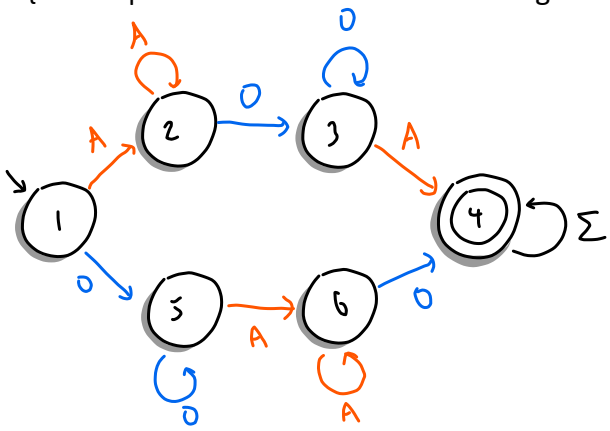
▸ **Aufgabe 1: Sprache ⇒ DEA** (2 + 2 + 2 Punkte)
Zeichnen Sie für jede der folgenden Sprachen L ⊆ Σ* einen *deterministischen* endlichen Automaten (DEA), der die Sprache L akzeptiert. Ihr Automat soll also alle Strings akzeptieren, die in der Sprache sind, dabei aber keinen String akzeptieren, der nicht in der Sprache ist.

1. L = {w ∈ Σ* | w enthält eine gerade Anzahl "A" und eine ungerade Anzahl "O"}

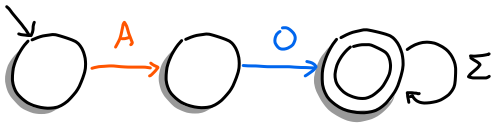


Zustand	A	O
1	gerade	gerade
2	ungerade	gerade
3	gerade	ungerade
4	ungerade	ungerade

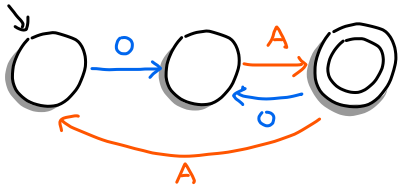
2. L = {w ∈ Σ* | w enthält sowohl den Teilstring "AO" als auch den Teilstring "OA"}



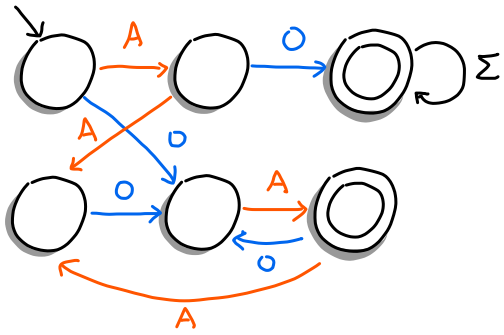
3. L = {w ∈ Σ* | w beginnt mit "AO" oder endet mit "OA" (oder beides)}



⇦ beginnt mit "AO"



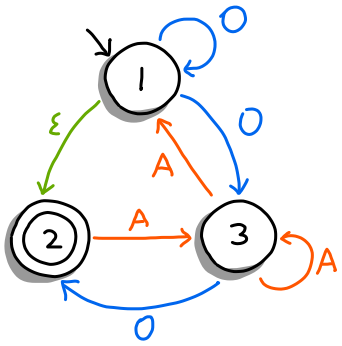
⇦ endet mit "OA"



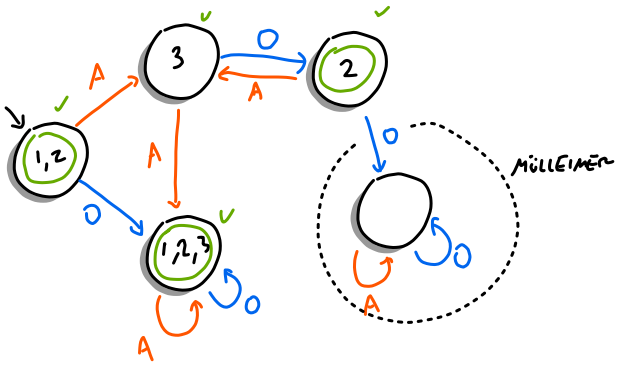
⇦ beginnt mit "AO" oder
endet mit "OA"

▸ **Aufgabe 2: ε-NEA ⇒ DEA** (4 Punkte)

Wandeln Sie folgenden ε-NEA in einen *deterministischen* endlichen Automaten (DEA) um:



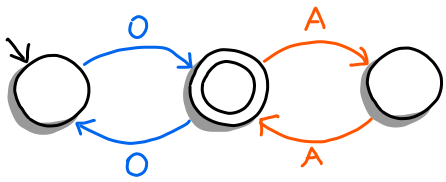
Markieren Sie eindeutig den Startzustand und alle akzeptierenden Endzustände Ihres DEA! Beschriften Sie außerdem jeden Zustand Ihres DEA mit der ihm entsprechenden Menge erreichbarer Zustände des ε-NEA! Dabei darf es für jede Teilmenge von {1, 2, 3} höchstens einen Zustand im DEA geben, der mit dieser Teilmenge beschriftet ist.



▸ **Aufgabe 3: Automaten ⇒ reguläre Ausdrücke** (2 + 2 + 2 Punkte)

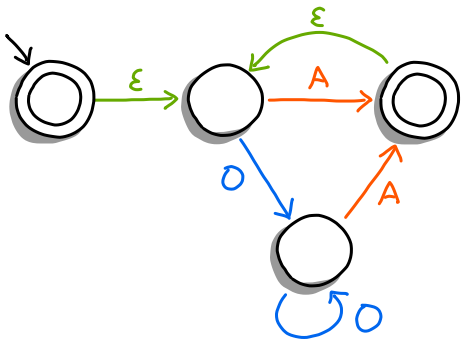
Geben Sie zu jedem der folgenden endlichen Automaten einen regulären Ausdruck an, der genau die Sprache definiert, die der Automat akzeptiert:

1.



O(OO|AA)*

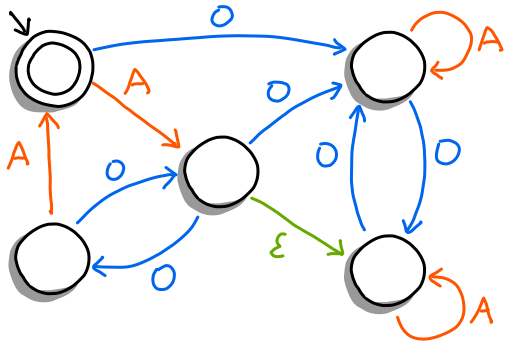
2.



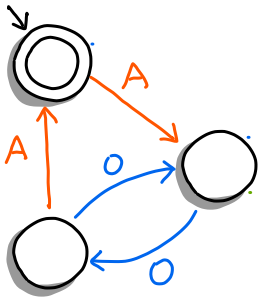
(A|OO*A)*

oder einfacher:
(O*A)*

3.



nach Entfernen unproduktiver Zustände:



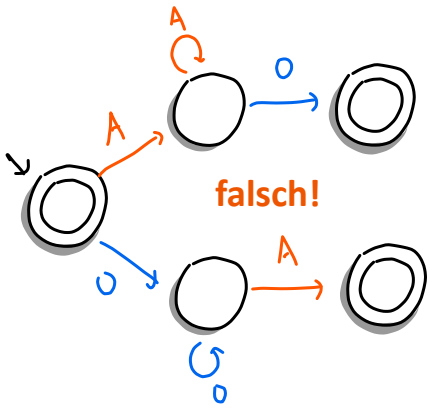
(AO(OO)*A)*

▸ **Aufgabe 4: reguläre Ausdrücke ⇒ DEA** (2 + 2 Punkte)

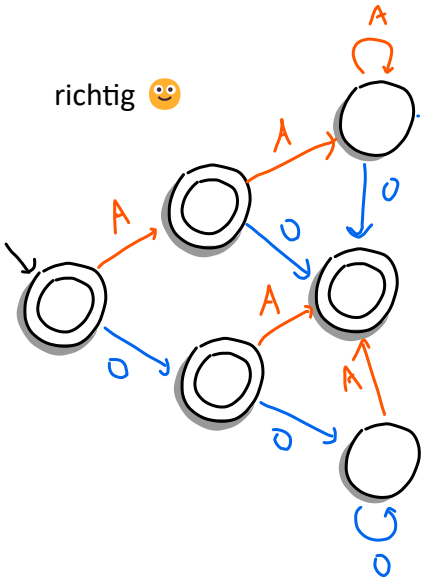
Zeichnen Sie zu jedem der folgenden regulären Ausdrücke einen *deterministischen* endlichen Automaten (DEA), der genau die von diesem regulären Ausdruck definierte Sprache akzeptiert:

1. ε|A*O|O*A
2. (AO|OA)*O

zu 1.:

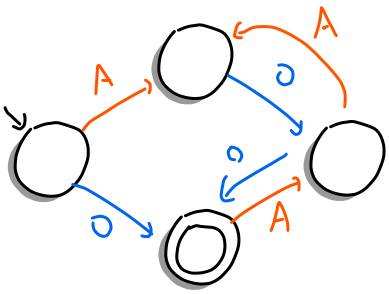


ε|A⁺O|O⁺A



ε|A*O|O*A

zu 2.



(AO|OA)*O

Kontextfreie Grammatiken

Context Free Grammars = CFG

$\langle N, T, S, P \rangle$

$N \cap T = \emptyset$
$S \in N$

Die Produktionsregeln in P lassen sich als $X \rightarrow w$ mit $X \in N$ und $w \in (N \cup T)^*$ schreiben

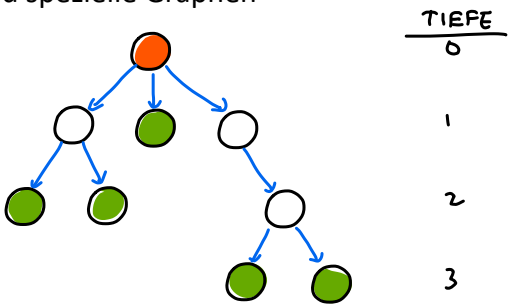
"Produktionsregeln" = "Produktionen" = "Ableitungsregeln"

Produziert werden Strings $\in T^*$

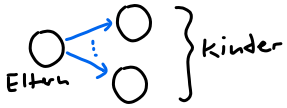
Achtung: Es darf nur endlich viele Produktionsregeln geben!

▸ Einschub: "Bäume" in der Informatik

Bäume sind spezielle Graphen



- Graph: Menge von Knoten, die mit Kanten verbunden sind
- Knoten: 
- Kanten: 
- Elternknoten zeigt auf Kindknoten; Kindknoten haben (maximal) einen Elternknoten.



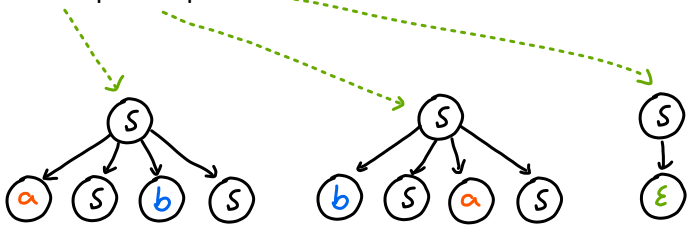
- Wurzel: Knoten ohne Elternknoten 
- Baum: hat genau eine Wurzel
- Blätter: Knoten ohne Kindknoten 
- innere Knoten: alle anderen Knoten, die weder Wurzel noch Blätter sind
- Tiefe eines Knotens v = Anzahl der Knoten auf dem Weg von der Wurzel bis zu v. Tiefe der Wurzel = 0.
- Höhe eines Baums = maximale Tiefe der Blätter im Baum

▸ Ableitungsbaum

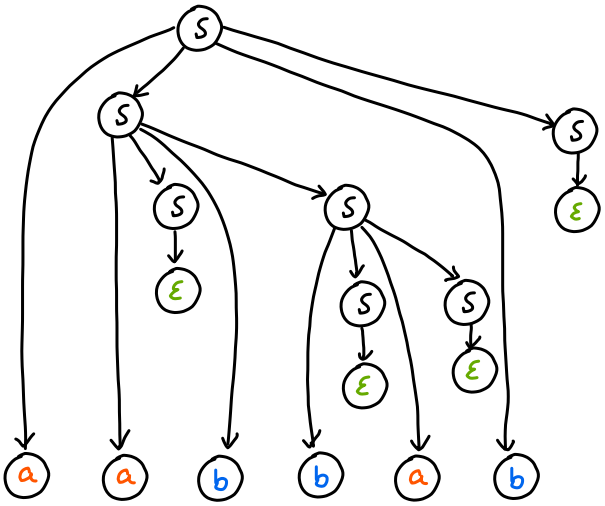
An der Wurzel steht das Startsymbol.
An den inneren Knoten des Baums stehen Nichtterminale.
An den Blättern des Baums stehen Terminale oder ε
Ein Knoten X hat Kindknoten $c_1, c_2, \ldots, c_n$, wenn es eine Produktionsregel $X \rightarrow c_1 c_2 \ldots c_n$ gibt.

▸ **Beispiel:**

S → aSbS | bSaS | ε



Ableitungsbaum für w = aabbab



Wenn ein String w in der von der Grammatik erzeugten Sprache ist, dann gibt es (mindestens) einen Ableitungsbaum für w.

▸ **Knobelaufgabe:**

Welche Sprache wird durch die folgende CFG erzeugt:

S → aSbS | bSaS | ε

Backus-Naur-Form

Nichtterminale in spitzen Klammern: ⟨Nichtterminal⟩
Strings von Terminalen in Anführungszeichen: "Terminale"
Statt → verwenden wir ::= (weil das Standardzeichen sind)

▸ Beispiel 1: arithmetische Terme

für (teilweise) geklammerte Rechenausdrücke, wie z.B. (x + y · z) · (z + x + y) + x

⟨Term⟩ ::=
  ⟨Term⟩ "+" ⟨Term⟩ |
  ⟨Term⟩ "·" ⟨Term⟩ |
  "(" ⟨Term⟩ ")" |
  "x" | "y" | "z"

⇒ wenn ich einen Ableitungsbaum von einem Term habe, kann ich entlang dieses Baumes, von unten nach oben, den Term auswerten. (wobei ich aufpassen muss, dass ich z.B. Regel "Punkt vor Strich", d.h. · hat Priorität von +, einhalte).

▸ Beispiel 2: reguläre Ausdrücke

z.B. für Σ = {"A", "N", "S"}

⟨regExp⟩ ::=
  "ε" | "Ø" | "A" | "N" | "S" |
  ⟨regExp⟩ ⟨regExp⟩ |
  ⟨regExp⟩ "|" ⟨regExp⟩ |
  ⟨regExp⟩ "*" |
  "(" ⟨regExp⟩ ")"

⇦ mit CFG können wir die Syntax (d.h. die Form, nicht die Semantik = Bedeutung) einer (formalen) Sprache definieren.